

周报temp

5.24

5.25

5.26

5.27

5.28

5.29

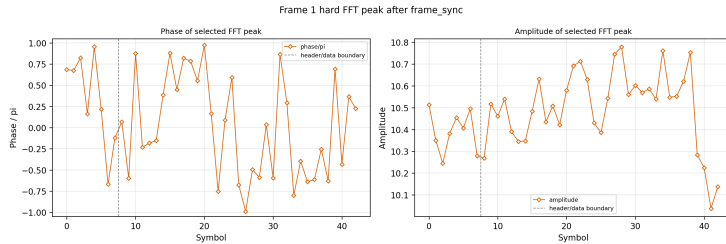
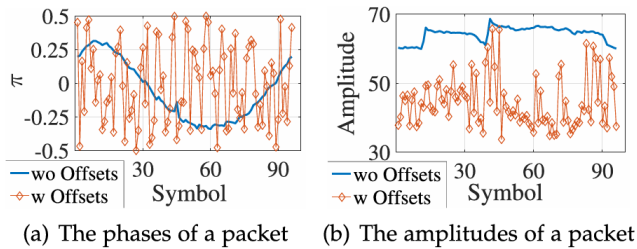
5.24

目前先不管编码层，先从如何选中更加正确的FFT PEAK的角度进行优化。所以之前针对payload的groundtruth/数据集构建粒度太粗了，因此还要再新增一个消息发布机制，负责通过grlorasdr解码链把每一个frame选中的fft bin topk结果记录下来。

目前已经把FFT bin选择的gt记录下来了。

然后我又发现一个问题就是如果直接复用grlorasdr的framesync，然后拿这里的发布数据去做**硬件缺陷特征追踪**的话，是不恰当的，因为framesync已经做了很多同步的工作了，像STO CFO这些引起的相位差可能更加不明显了，直观上并不适合直接拿来建模分析（但是这也只是猜想，有待进行实验证明）

观察一下grlorasdr导出的fft peak相位和幅度特征：按照我的阅读，这个解码链应该导出来去过offsets的数据了



这一点也不平滑啊，和左边的Hi2lora相比这应该还是没有去offsets的版本吧。

5.25

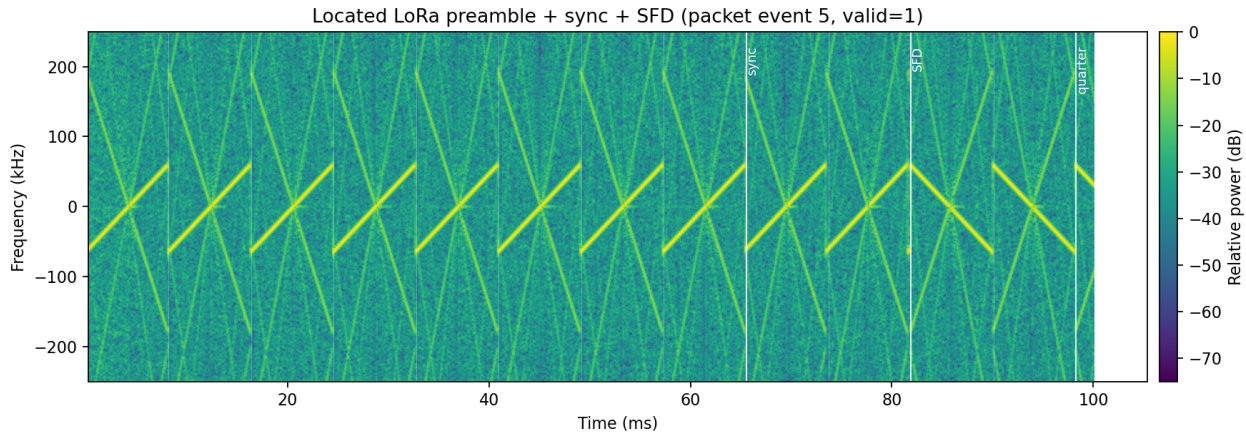
重写了检测逻辑，主要是加入了滑动窗口的设计，流程如下：

raw complex64 IQ：

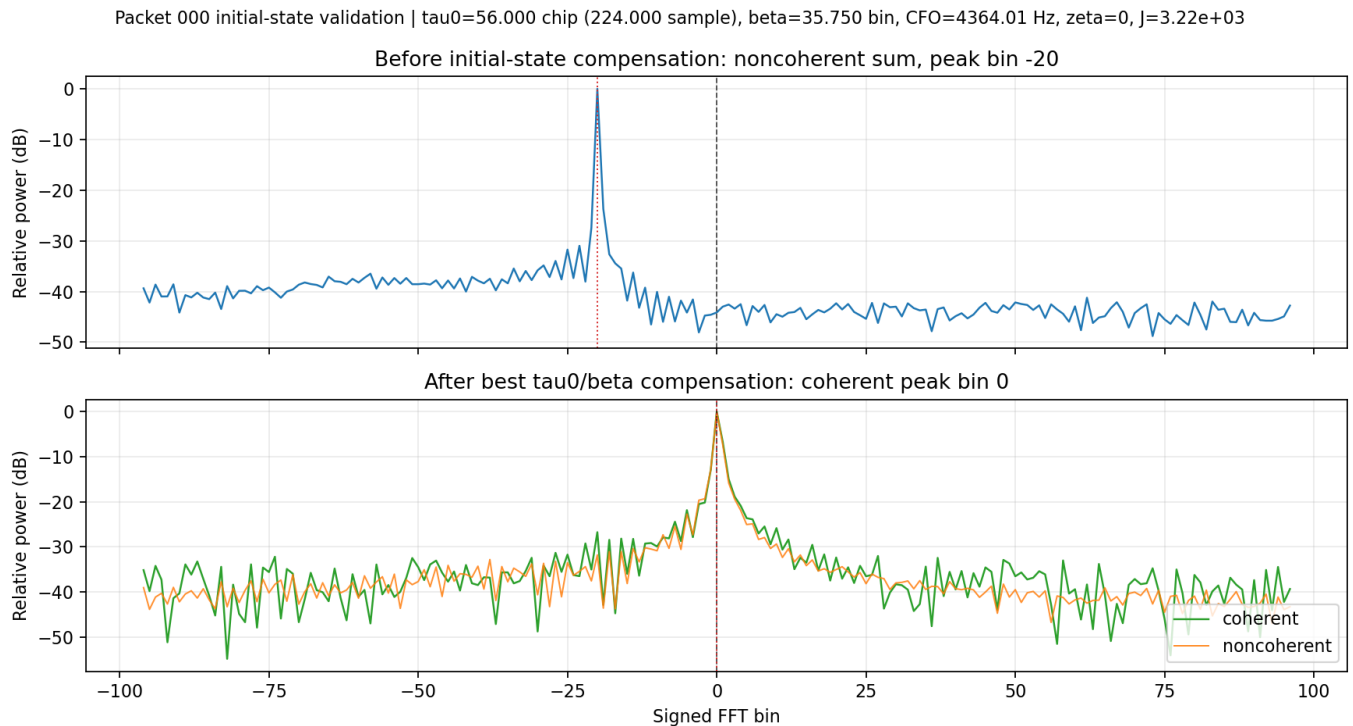
- > 不先降采样，保留 os_factor 过采样 chirp 长度
- > 每个滑窗包含 win_chirps 个 chirp
- > 每个 chirp 分别 dechirp + FFT
- > 对窗口内所有 chirp 的 FFT 能量按 bin 累加
- > 取累加能量最大的 peak bin
- > 连续窗口 peak bin 稳定超过 min_periodic_peaks 就报检测。

5.26

粗检测还不够，需要结合SFD进行一个完整的帧定界才可：



这里就没有降采样再DTFT了，总之现在粗帧定界做差不多了。



目前的初始状态估计也做得尚可，这两幅图代表一个数据包内所有前导码dechirp+FFT后进行相干叠加的结果，可以看到补偿了STO_0和CFO后，这个bin基本上集中在了bin0处。

tau0_chip 是前导码第一个 upchirp 开头位置的初始 STO 估计，也就是以 $\text{located_preamble_start_sample}$ 作为参考的第 0 个前导码符号状态。

beta_bin 是这个包的常值 CFO 估计，默认认为整个包基本共用同一个 CFO，所以它不是“只属于前导码开头”或“只属于 payload 开头”，而是包级频偏参数。

zeta 是SFO 漂移项，描述 STO 随符号编号怎么慢慢变化。当前默认 $\text{--zeta-span } 0$ ，所以一般就是 0，等于暂时不估 SFO。

payload 即将开始位置的 STO 不是直接用 tau0_chip ，而是由模型外推出来：

$$\text{payload_sto_chip} = \text{tau0_chip} + s_{\text{payload}} * (M - 1) * \text{zeta}$$

```
payload_sto_sample = R * payload_sto_chip
```

其中：

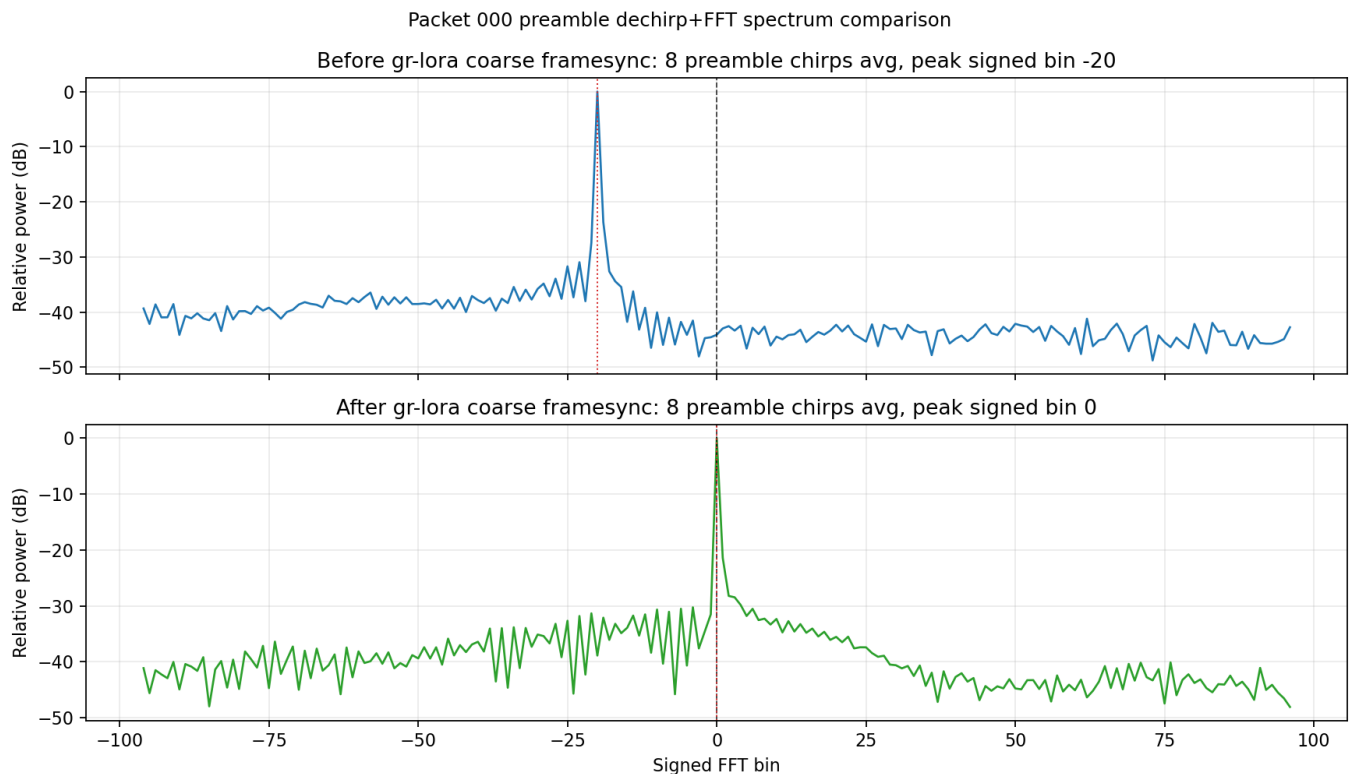
```
s_payload = preamble_len + 4.25
```

现在有了这些估计，我们再根据SFD和前导码的性质：bin_offset 估计：它只用整数/亚 bin 峰值位置，假设 $\text{zeta}=0$ ，用 $\text{up_peak} \approx \text{beta} - \text{tau}$ 、 $\text{down_peak} \approx \text{beta} + \text{tau}$ 解出一组粗 CFO/STO，来做一下横向对比，看一下估计精度如何。

结果发现，前导码only的优化和结合SFD的粗估计，两者的结果大相径庭，感觉是因为前导码only还是难以处理CFO和STO在频率bin偏移上的一个耦合性质：

这组结果的作用不是替代相干优化，而是用 SFD 打破只看 upchirp 时的 CFO/STO 耦合。当前 `0_0_0_10_14_8.bin` 上可以看到：bin-offset 解析估计给出的 STO 接近 0 chip、CFO 接近 signed bin -20；而只用 upchirp 的相干优化会找到另一组 `beta/tau`，但它们仍满足 `beta - tau` 接近 -20，因此也能把前导码峰拉回 bin0。

想来想去还是把这个初始状态估计模块丢了，这个算得太扯了，最后决定复用grlorasdr的模块：但我这里留了一个点没有调整，因为grlorasdr对于STO的小数估计是基于FFT BIN的偏移，并无法发挥我们过采样的优点，**其实这里时间延迟允许可以在时间域上进行滑动然后得到最优的STO**



这个图结果明显比之前前导码only要强悍，主峰如此尖。

目前输出：

grlora_sfo_hat: gr-lora_sdr 语义里的 SFO 漂移量, 近似表示每经过 1 个 LoRa symbol, STO 会累积漂移多少 chip。

grlora_payload_sto_frac_est: 把 STO 从 preamble 起点推进到 payload 起点后的 fractional STO, 也就是 payload 开始位置的 STO_frac 估计。

grlora_sfo_cum_initial: gr-lora_sdr 进入 payload 后用于后续 sample insertion / deletion 的初始累计小数偏移。

grlora_fine_payload_start_sample: 综合 CFO_int 等效 timing、netid_offset、payload 处 STO_frac 后得到的 payload 起始采样点。

总结一下目前实现的检测+帧定界流程:

▼ 流程

第一阶段：弱前导码检测

这里没有检测 sync word / 网络 ID。

它只做：

- 滑动窗口扫描；
- 每个窗口内多个 chirp dechirp+FFT；
- 累加 FFT 能量；
- 判断 peak bin 是否周期性稳定出现；
- 输出 coarse detection event。

所以这一层只回答：“这里像不像有 LoRa upchirp 前导码”。

第二阶段：帧定界 `frame_locator.py`

这里有用 sync word 做筛选。

逻辑是：

- 在 coarse start 附近搜索候选 preamble 起点；
- 假设前导码后面紧跟两个 sync word；
- 根据 `sync_word=0x34` 算出期望符号：
 - `sync1_expected_bin`
 - `sync2_expected_bin`
- 实测：
 - `sync1_bin`
 - `sync2_bin`
- 计算：
 - `sync1_distance`
 - `sync2_distance`
- 如果距离超过 `--frame-sync-bin-tol`，`frame_valid` 就会变成 0。

所以 `frame_valid` 已经包含 sync word 检查。

第三阶段：gr-lora 风格 framesync 验证 `grlora_frame_sync.py`

这里有更接近 gr-lora_sdr 的 netID 检查。

它会输出：

- `grlora_netid1_est`

- `grlora_netid2_est`
- `grlora_netid_offset`
- `grlora_netid_valid` (这个是否还有存在必要?)
- `grlora_sync1_distance`
- `grlora_sync2_distance`

而且最终:

```
1  grlora_framesync_valid =
2      前导码校正后都落 bin0
3      && sync1_distance == 0
4      && sync2_distance == 0
5      && grlora_netid_valid
```

所以 `grlora_framesync_valid` 比 `frame_valid` 更严格。

一句话: 弱检测本身不筛 sync word / 网络 ID; 帧定界阶段筛 sync word; `grlora_framesync` 验证阶段进一步筛 netID, 并输出 `grlora_netid_valid`。

做完以上工作应该开始开始payload部分的特征建模了:

- 先要有groundtruth (这个怎么拿)
- 然后在有groundtruth的基础上, 去给每个符号去一下STO 观察相位和幅度的变化趋势

有个更有意思的现象: `lora_file_RX.py` 解出来的 payload 计数是 `0x13, 0x15, 0x17, 0x19, 0x1a, 0x1b, 0x1c`, 看起来早期漏的是中间隔着的几个包, 而不是尾部读不完。这更像原始 `frame_sync` 在某些帧同步判定处退回了 `DETECT`, 不是导出脚本筛掉了。

gcd我的检测逻辑已经达到弱包检测的目的了吗, 已经比`grlorasdr`的牛点了, 那看来只能接着用, 有一些只有一部分前导码的就丢掉不管呗。

5.27

目前STO_frac的对齐依赖的是2N的FFT, 再通过三点插值来求的chip级别偏移; 但是在过采样的场景下, 这里或许可以用优化方法替代, 移动窗口直到peak最尖最高。

目前对STO处理策略: 不是一个固定 offset 一用到底, 而是先用 `m_sto_frac` 对齐起点, 再用 `sfo_cum` 在整个包内动态微调符号边界。

显示的payload header 长度是8 chirps, 目前实现了framesync之后的FFT demod:

▼ header长度

explicit header 模式:

有 PHY header

header 部分固定先按 8 个 LoRa symbols 解

implicit header 模式:

空口里没有 explicit PHY header

payload_len / CR / CRC 由接收机预先配置

因此不需要解这 8 个 header symbols

目前使用: 0_0_0_10_14_16.bin 跑了一遍

- 输入 framesync-valid 候选: 11 个 (就算在检测阶段能够绘制出STFT图的包, 也未必framesync valid)
- header checksum 通过: 11/11
- 每个包解出: payload_len=33, CR=1, has_crc=1, LDRO=0
- 每包导出: 8 header + 35 payload = 43 个 symbol peak

但是只是过了检测的话是有13个包的, 所以其实有3个数据包没有通过framesync,

▼ framesync_valid 判断

`grloro_netid_valid` 本质上是在回答：

- 1 这两个 sync word symbol 是否整体符合 LoRa sync word, 并且允许同步残差造成的共同 bin 偏移?

而 `grloro_sync1_distance == 0`、`grloro_sync2_distance == 0` 回答的是：

- 1 这两个 sync word symbol 是否分别精确落在期望 bin?

区别就在“是否允许共同 offset”。

LoRa 的 sync word 由两个符号组成, 比如 `0x34` 会转成：

- 1 `sync1_expected = 24`
- 2 `sync2_expected = 32`

理想情况下当然应该看到：

- 1 `netid1 = 24`
- 2 `netid2 = 32`

但实际同步里, 如果还残留一点整数 STO / CFO 耦合, 两个 sync word 可能一起偏移, 比如：

- 1 `netid1 = 25`
- 2 `netid2 = 33`

这时候：

- 1 `sync1_distance = 1`
- 2 `sync2_distance = 1`

如果你要求两个 distance 都是 0, 这个包就被判死了。

但从结构上看, 它其实很合理, 因为两个 sync word 保持了同样的相对关系：

- 1 `netid2 - netid1 = 33 - 25 = 8`
- 2 `expected2 - expected1 = 32 - 24 = 8`

所以 `gr-lora_sdr` 的逻辑是允许一个共同偏移：

```
1 netid_offset = netid1_est - sync1_expected
2
3 abs(netid_offset) <= 2
4 and mod(netid2_est - netid_offset, N) == sync2_expected
```

例子：

```
1 netid1_est = 25
2 netid2_est = 33
3 sync1_expected = 24
4 sync2_expected = 32
```

那么：

```
1 netid_offset = 25 - 24 = 1
2 netid2_est - netid_offset = 33 - 1 = 32
```

所以 `grlora_netid_valid = 1`。

这比单纯要求：

```
1 sync1_distance == 0
2 sync2_distance == 0
```

更合理，因为它承认了一个事实：**sync word** 的两个符号会受到同一个残余同步偏移影响。我们真正关心的不是它们有没有绝对精确命中，而是它们是否作为一组保持了正确的 sync word 结构。

所以建议后面收敛成：

```
1 framesync_valid =
2     preamble_corrected_to_bin0
3     && grlora_netid_valid
```

而：

```
1 grlora_sync1_distance
2 grlora_sync2_distance
```

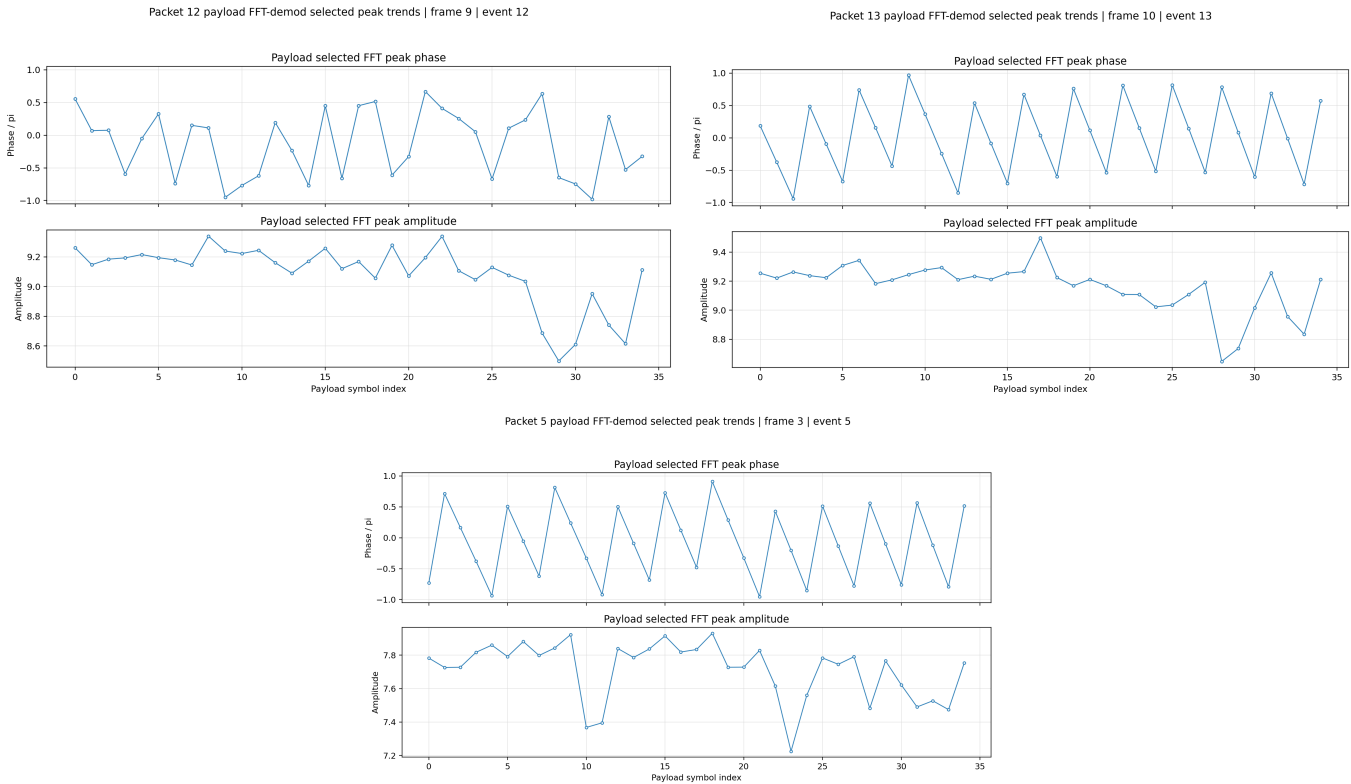
保留为调试字段，用来看残余 offset 有多大。

所以，其实grlorasdr like的 netid valid判断显然更鲁棒，syncword和expected存在相同offsets要比严格的bin落在同一位置来得合理。

FFT demod逻辑写完之后，如何验证选的peak是正确的，以及目前这个选peak逻辑是否进行去offsets，符号的幅度和相位特征如何？

一个bin文件里面的包，除了packetID不一致外，其他部分应该是一样的，可以做一下**内部检验**（想把NETID就算没有valid的也通过一下，进入FFT demod流程，看看准不准）

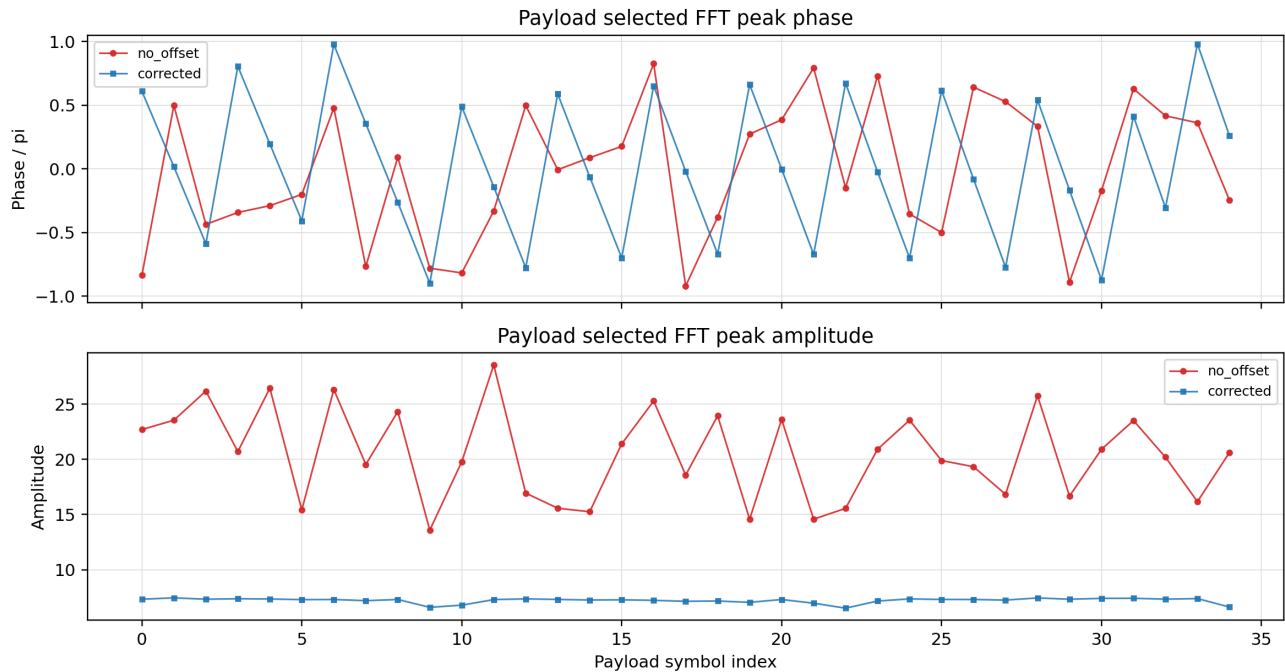
在 11 个最终有效帧里，payload symbol 5..29 的 raw FFT bin 完全一致；变化集中在 0..4 和 30..34。这很符合“payload 里只有 packetID 不同”的直觉，因为 packetID 会经过 whitening / Hamming / interleaving，变化不会只落在一个 LoRa symbol 上；尾部变化也很可能是 CRC 跟着 packetID 变化导致的。



选了三张还比较有代表性的图，感觉幅度丑的拉稀，但是相位其实除了左上角（少数）其他的都是和右上角和下面那张图一样，上下摆动，怎么说呢还是比较有规律的，现在要来检查一下，绘制这图之前，去offsets去得彻不彻底。毕竟王帅那个是建立在：完全去OFFSETS的基础上做的，现在说实话我也去差不多了，感觉也就频率跳变那里没去得了？

补一个实验，用我们自己的原型系统做一个没有去offsets的对比图：

packet_id=2 | frame_id=1 | event_id=2 | no_offset vs corrected



corrected是去了offsets的，另一个没有哈

用已有同步链确定这个 packet 在哪里，但在 payload FFT 特征提取时不做 offsets compensation。

有个现象：首先是corrected后这个相位明显有wrapped痕迹、另一个就算这个corrected的幅度怎么这么低？

这个现象本质上是一个很基础但很容易坑人的点：

FFT 输出的“幅度”不是物理幅度本身，而是一次相干求和的结果；你拿多少个采样点做 FFT，峰值幅度就会跟着变。

▼ 解释

FFT 的峰值幅度本质上是相干求和。假设 dechirp 后的信号刚好是一个落在第 k_0 个 FFT bin 上的纯 tone:

$$x[n] = Ae^{j2\pi k_0 n/N}, \quad n = 0, 1, \dots, N-1$$

那么在第 k_0 个 bin 上, FFT 输出是:

$$X[k_0] = \sum_{n=0}^{N-1} Ae^{j2\pi k_0 n/N} e^{-j2\pi k_0 n/N}$$

两项相位刚好抵消, 所以变成:

$$X[k_0] = \sum_{n=0}^{N-1} A = AN$$

因此:

$$|X[k_0]| = AN$$

这就是关键: 如果 FFT 没有归一化, 峰值幅度会和 FFT 长度 N 成正比。

你的 corrected 流程大概是 chip-rate FFT:

$$N = 2^{\text{SF}}$$

如果 $\text{SF} = 10$, 那么 $N = 1024$ 。

而 no-offset 脚本之前用了过采样整符号 FFT:

$$N_{\text{os}} = 2^{\text{SF}} \cdot \text{os_factor}$$

如果 $\text{os_factor} = 4$, 那么 $N_{\text{os}} = 4096$ 。

所以即使物理信号幅度 A 完全一样, 两个 FFT 峰值幅度也可能差约 4 倍:

$$\frac{|X_{\text{chip}}[k_0]|}{|X_{\text{os}}[k_0]|} \approx \frac{A \cdot 1024}{A \cdot 4096} = \frac{1}{4}$$

这就是你看到 no-offset 幅度看起来大很多的基础原因。

但这不代表 no-offset 更好。因为真正重要的不是峰值绝对幅度, 而是能量是否集中到一个 bin。更公平的指标是:

$$\text{peak_energy_ratio} = \frac{|X[k_{\text{max}}]|^2}{\sum_{k=0}^{N-1} |X[k]|^2}$$

如果 no-offset 没补偿 CFO/STO/SFO, 能量可能扩散到很多 bin; corrected 后虽然 FFT 长度短、绝对峰值低, 但如果大部分能量压回主峰, 那么:

$$\text{peak_energy_ratio}_{\text{corrected}} > \text{peak_energy_ratio}_{\text{no offset}}$$

这才说明 corrected 更干净。

所以结论是：直接比较 $|X[k_{\max}]|$ 不公平，因为 FFT 长度不同；应该比较归一化幅度、峰值能量占比、peak margin。

比如可以比较：

$$\text{normalized_amp} = \frac{|X[k_{\max}]|}{\sqrt{\sum_{k=0}^{N-1} |X[k]|^2}}$$

或者：

$$\text{peak_margin_dB} = 20 \log_{10} \frac{|X[k_1]|}{|X[k_2]|}$$

其中 k_1 是最大峰 bin, k_2 是次大峰 bin。

你现在那个现象可以简化理解成一句话：

过采样 FFT 用更多采样点参与相干求和，所以绝对幅度天然更大；但 corrected 的能量占比更高，说明它的峰更集中，反而更符合补偿有效的预期。

▼ 那过采样 FFT 到底好在哪里？

它的好处主要在这几件事：

第一，它保留了 fractional STO 信息。

chip-rate FFT 是每个 chip 只取一个样点。如果你的取样点偏了，比如 STO_frac 不是 0，那么你可能刚好取在不理想的位置。过采样数据里每个 chip 有多个样点，你可以观察不同取样相位下 FFT peak 怎么变。

第二，它方便观察频谱泄露。

如果 CFO/STO/SFO 没补偿好，dechirp 后的 tone 不再干净落在单个 bin 上，能量会扩散到邻近 bins。过采样视角可以更清楚地看这种扩散形态，尤其适合做诊断。

第三，它方便做 fractional offset 搜索。

比如你可以尝试不同的 fractional STO：

$$\tau \in [-0.5, 0.5] \text{ chip}$$

然后看哪个 τ 下：

$$\frac{|Z[k_{\max}]|^2}{\sum_k |Z[k]|^2}$$

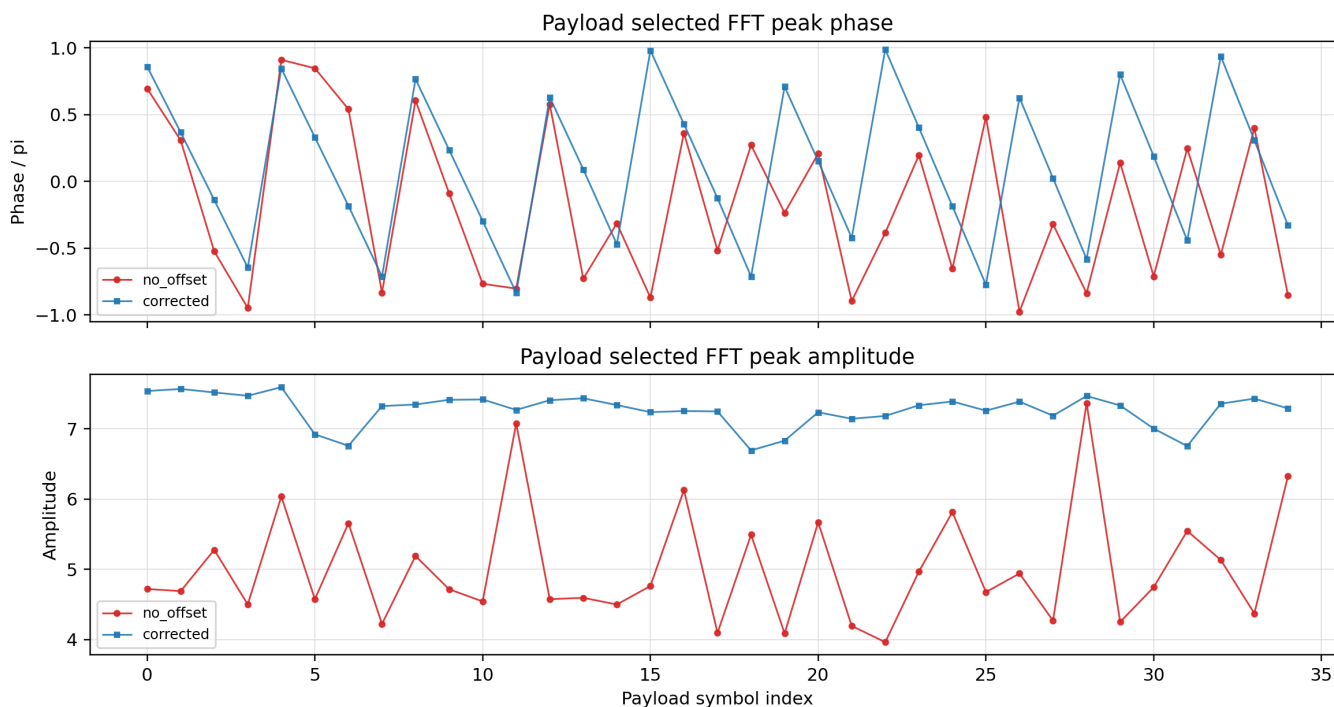
最大。这个比直接 chip-rate 一刀切更灵活。

第四，它可以辅助做更细的频偏估计。

但这里要注意，真正让频偏估计更细的通常不是"直接过采样 FFT"，而是 zero-padding 或者峰值插值，比如 parabolic interpolation、Quinn estimator、RCTSL/Bernier 这类方法。

暂时为了方便，采用 chip-rate FFT

packet_id=3 | frame_id=2 | event_id=3 | no_offset_chirate vs corrected



可以看出幅度确实有长进，但是相位感觉提升不大，今天先暂时不分析问题出在哪个xx offsets没消除干净了，但是有一点一定要注意：

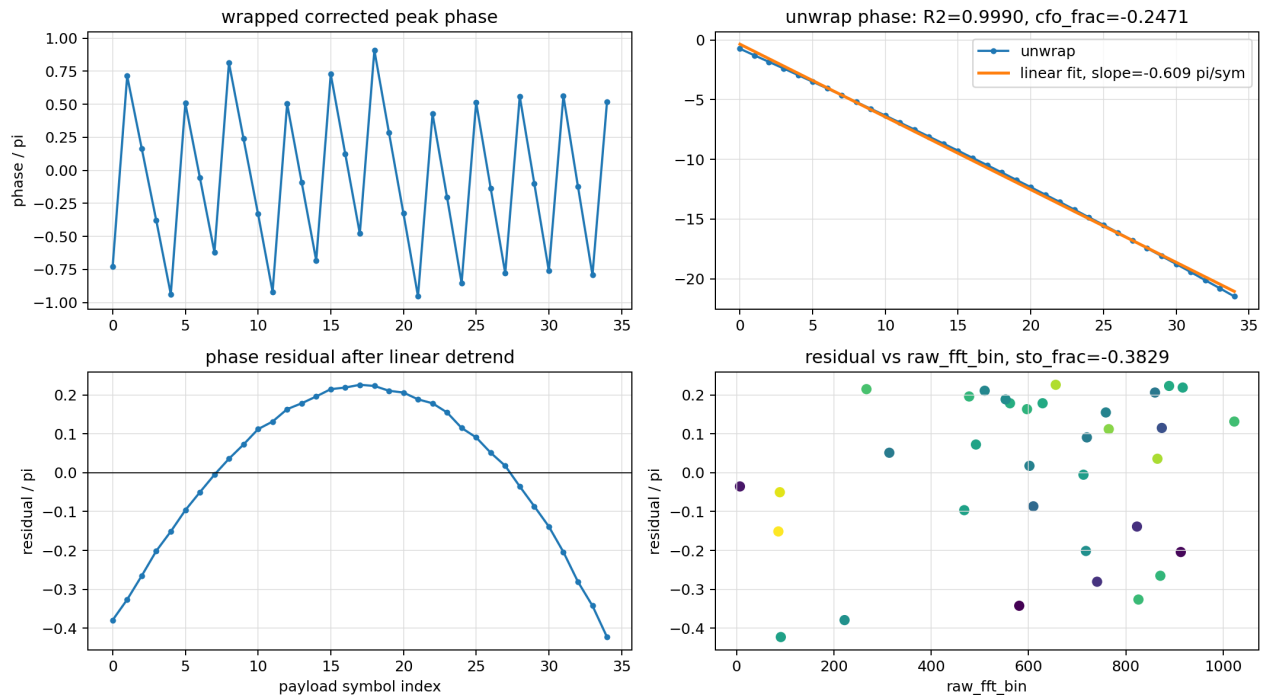
offline corrected 其实复刻的是 gr-lora_sdr 的接口语义，**frame_sync** 输出的是"去掉 STO、但还带 CFO 的 chip-rate symbol"，**fft_demod** 用本地 downchirp 消 CFO 对 FFT 聚峰有帮助，但不会把每个 symbol 的全局 CFO 累积相位清零。所以拿 **peak_phase** 画包内趋势，自然会看到线性 wrapped sawtooth。

▼ 那我一开始估计出全局CFO之后，然后对着整个包进行一个旋转补偿，可以实现相位差消除吗

换言之：STO/SFO/CFO 用来让 FFT peak 聚起来，但没有把每个 payload symbol 的全局 CFO 累积相位去掉

所以你看到的蓝线锯齿，主要是 wrapped phase 把线性相位漂移折叠了。

展开一看果真如此：



▼ 图片内容解释

四个子图含义：

左上：wrapped corrected peak phase

这是直接画 CSV 里的 $\text{peak_phase} / \pi$ ，范围被限制在 $[-1, 1]$ ，也就是 $[-\pi, \pi]$ 。

它看起来锯齿状，是因为相位本来在持续往下漂，一旦低于 $-\pi$ 就会跳回 $+\pi$ 。所以这个图不能直接说明 phase 随机，它只是 wrapped phase 的视觉效果。

右上：unwrap phase + 线性拟合

这里对左上的相位做了 $\text{np.unwrap}()$ ，把跳变接回连续相位。

结果非常关键：

$$R^2 = 0.9990$$

$$\text{slope} = -0.609 \pi / \text{symbol}$$

$$\text{cfo_frac} = -0.2471$$

这说明 corrected phase 几乎是一条直线。也就是说，锯齿的主因是 包内残余的线性相位漂移，高度符合 residual/global CFO phase 的特征。

$\text{cfo_frac} = -0.2471$ 和斜率同号同量级，说明当前 corrected FFT 虽然用了 CFO_frac 让峰聚起来，但没有把每个 symbol 的全局 CFO 累积相位完全归一掉。

左下：phase residual after linear detrend

这里把右上的线性趋势减掉，剩下残差。

残差不是随机噪声，而是一个比较平滑的弯曲趋势，幅度大概在：

$$-0.4\pi \text{ 到 } +0.22\pi$$

这意味着除了主线性的 CFO 相位项，还有一点低阶弯曲项。这个更像 residual SFO、采样时钟漂移、边界推进误差，或者相位参考点没有完全一致导致的二次/慢变项。

右下：residual vs raw_fft_bin

这是把“去掉线性漂移后的相位残差”画到 raw_fft_bin 上。

目的就是验证你说的那个假设：

$$\text{residual STO} \propto \text{symbol bin}$$

如果 residual STO 很明显，理论上 residual phase 会和 FFT bin 有比较明显的线性关系。

但这张图里点比较散，没有一条很强的直线。所以对 packet 005 来说，STO × bin 不是第一主因。主因还是右上那条线性相位漂移。

5.28

✅ 分析昨天的四联图（已经分析完了放到grlor同步策略文档里了）

左下角那个图简单点说就是相位随时间二次变化，说明频率偏移是线性变化的

▼ SFO结论

推导出的 SFO 相位误差是：

$$\Psi_{\text{SFO}}(q, r; s_q) = \frac{a^2 - 1}{2N} r^2 + \left[q(a^2 - a) + (a - 1) \left(\frac{1}{2} + \frac{s_q}{N} \right) \right] r + \Gamma_q(s_q)$$

其中：

$$\Gamma_q(s_q) = \frac{q^2 N (a - 1)^2}{2} + \frac{q N (a - 1)}{2} + q s_q (a - 1)$$

这里前三项里面，前两个含 r 的部分：

$$\frac{a^2 - 1}{2N} r^2 \quad \text{和} \quad \left[q(a^2 - a) + (a - 1) \left(\frac{1}{2} + \frac{s_q}{N} \right) \right] r$$

主要描述的是一个 chirp 内部的相位形状变化。它们会影响 dechirp+FFT 后能量是否集中，比如 peak energy ratio、peak margin、FFT 泄漏。

但 $\Gamma_q(s_q)$ 不含 r ，它是当前第 q 个 symbol 的**公共相位**。它不会明显改变 FFT peak 的位置，但会直接进入你画的 selected peak phase。

只能说虽然左下角那个图是个类似二次曲线，但是其一可以从SFO相位误差里看出，含 q 不含 r 的相位差里面，首先是 q 方的系数是正数和曲线开口向下对不上，其次是 q 的一次项里有 s_q ，这意味着对称轴会随着符号变化而变化的，不可能这么平滑，**所以不能简单归结为SFO残余**

这或许已经是**环境和硬件两者共同作用导致的结果**了，这里将其归结为**packet-level common phase drift**

所以我现在会把左下角解释成：

corrected selected peak phase 中，去掉线性 residual CFO 后仍存在的 packet-level low-order phase drift。

它可能包含 SFO 贡献，但不能单独称为 SFO 证据，感觉是混合了很多因素导致的结果

而右下角的图片说的是：去掉全局线性相位推进（CFO）之后，剩下的相位残差是否和 FFT bin 有关系？STO会带来一个和bin值线性相关的相位差,现在从右下角的图片可以看出残余误差和STO压根没关系，所以应该不是这个导致的

- ☐ 目前解码链粗检测之后的频率偏移是否进行了全局消除？
- ☐ framesync阶段，STO_frac的估计是否可以用窗口进行采样点级别移动而不是简单的FFT去进行更加精密的估计
- ☒ 解码链要再顺一下，framesync valid判断是在fft demod前还是后

▼ 回复

- ☐ run_weak_sync_chain.py
 - ☐ 1. weak preamble detection
 - ☐ 2. coarse frame locator
 - ☐ 3. gr-lora-style frame sync validation
 - ☐ -> 这里产生 **grlora_framesync_valid**（会检查sync word）
 - ☐ -> 同时输出 CFO_int / CFO_frac / STO / SFO / fine_payload_start 等
 - ☐ 4. 写 sync_chain.csv
 - ☐
- ☐ run_header_first_demod.py
 - ☐ 5. 读取 sync_chain.csv
 - ☐ 6. 默认只选择 grlora_framesync_valid == 1 的候选
 - ☐ 7. 对这些候选做 header-first FFT demod
 - ☐ 8. header decode
 - ☐ 9. **header_valid** 后继续 payload FFT demod

- ☒ unwrapped的数学原理是什么？2pi整数倍的相位旋转FFT能感知到吗？我艹我目前这个是实验观察不会是伪证吧nm

放心，大概率不是。因为unwrapped虽然并不是FFT感知出来的结果（是我们的假设，假设出现SAW状的相位变化是因为相位被限死在一圈内了），unwrap一般是可靠的。因为每个symbol之间相位真实变化没有超过半圈，unwrap不太容易接错。要是真担心这个出错，去掉CFO就好啦，可以看到去掉CFO左下角的曲线就很丝滑了

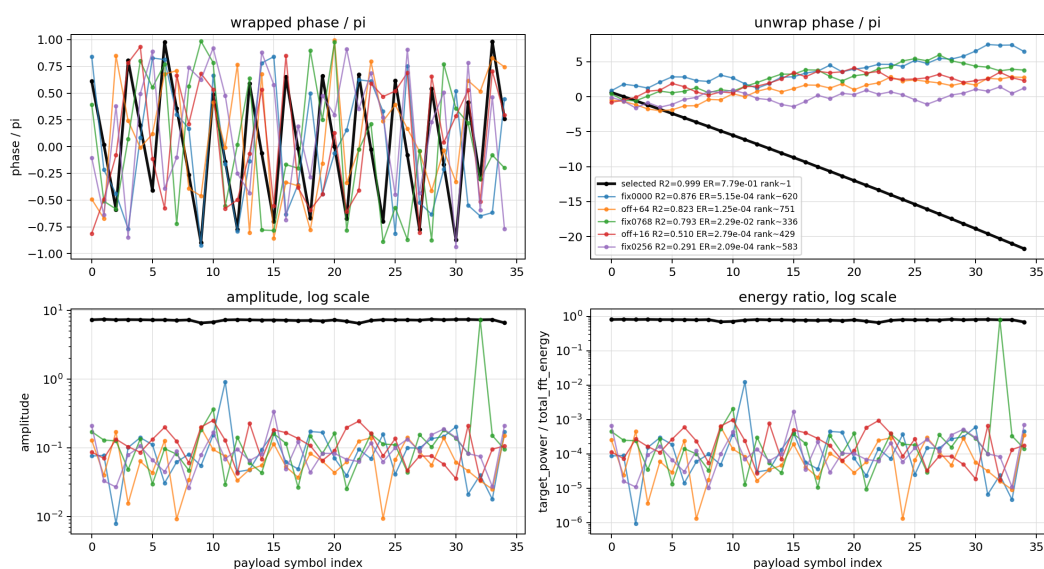
- [📎欧阳琛-进度汇报-2026-5.28.pptx](#)

- ☒ 我突然意识到一个问题，如果这些相位幅度的平滑特征，就算选错了FFT bin也存在会咋整，毕竟这

些硬件特征可能整个包内都有呢，请进行实验和相应四联图绘制，在FFT demod阶段（去除了offsets）选其他bin，绘制这个图，还会有这么好看的结果吗？

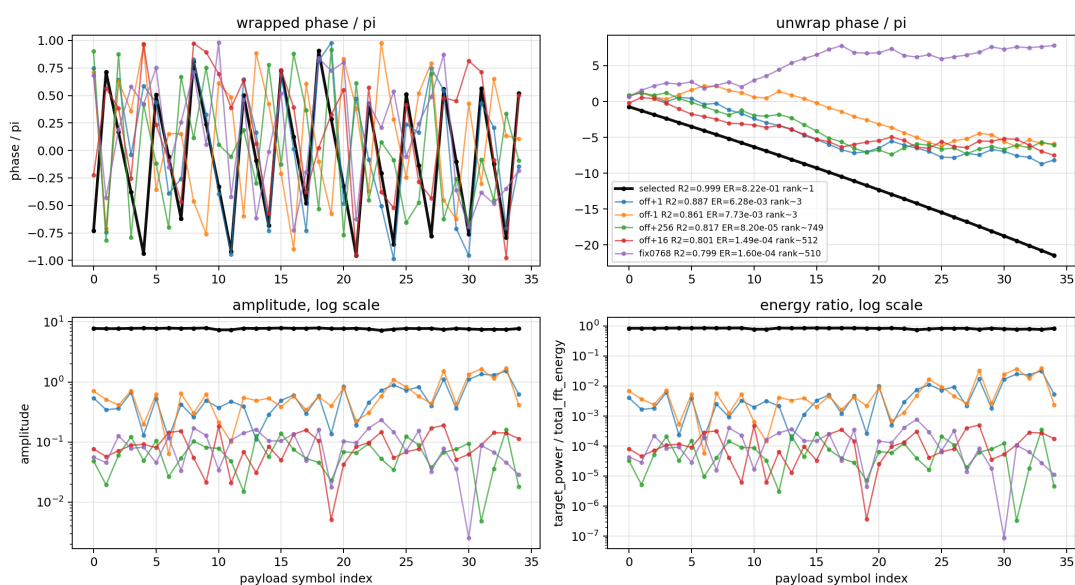
▼ 回复

Packet 2 corrected FFT wrong-bin control | event=2 | plotted wrong bins=highest R2



- ☐ 做了一下实验，我们故意选择wrong bin进行相位，幅度分析。选wrong bin的逻辑为：
- ☐ 选择相对于最高peak偏移固定值的wrong bin，或者fix 固定选择xx bin。
- ☐ 可以发现，选了wrong bin是无法呈现出类似groundtruth那样的幅度和相位变化特征的（低信噪比有待验证）

Packet 5 corrected FFT wrong-bin control | event=5 | plotted wrong bins=highest R2



- ☐ 但如果看这张图，发现出了gt以外，offset设置比较小的曲线的相位变化R2也很高，energy_ratio = 这个 bin 的能量 / 整个 FFT 所有 bin 的总能量 也不低。
- ☐ 所以更准确的结论应该是：

- ☐ phase smoothness 单独没有判别力 (R2) ;
amplitude / energy ratio / rank 才能把 wrong bin 区分出来;
phase smoothness 只能作为辅助一致性特征。
- ☐ 如果你只看右上角 unwrap phase, 有些 wrong bin 确实“不算太差”; 但一看下面两个能量图, 它们就明显露馅了。
- ☐ dechirp + FFT 本质上是在做一组正交匹配滤波。正确 bin 上信号相干叠加, 错误 bin 上信号相互抵消, 只剩噪声、泄漏和少量旁瓣。
- ☐ 所以正确 bin 才会同时表现出: 幅度高、能量占比高、相位可展开且连续
- ☐ 而 wrong bin 通常表现为: 幅度低、能量占比极低、相位不稳定
- ☐ 下面是数学分析

- ☐ 1. FFT demod 的数学本质: 正确 bin 相干叠加
- ☐ 假设第 k 个 payload symbol 在 dechirp 之后变成一个整数频率 tone:
 - ☐ $y_k[n] = A_k e^{j\theta_k} e^{j2\pi s_k n/N} + w_k[n], \quad n = 0, \dots, N-1$
- ☐ 其中:
 - ☐ s_k : 真实 FFT bin
 - ☐ A_k : 幅度
 - ☐ θ_k : 当前 symbol 的全局相位, 里面包含 CFO 相位推进、信道相位等
 - ☐ $w_k[n]$: 噪声和残余干扰
 - ☐ $N = 2^{\text{SF}}$
- ☐ 对任意 FFT bin b , FFT 输出是:

$$\input type="checkbox" Z_k[b] = \sum_{n=0}^{N-1} y_k[n] e^{-j2\pi bn/N}$$

- ☐ 代入:

$$\input type="checkbox" Z_k[b] = A_k e^{j\theta_k} \sum_{n=0}^{N-1} e^{j2\pi(s_k-b)n/N} + W_k[b]$$

- ☐ 关键就在这个求和项:

$$\input type="checkbox" \sum_{n=0}^{N-1} e^{j2\pi(s_k-b)n/N}$$

- ☐ 如果 $b = s_k$, 那么每一项相位都对齐:

$$\input type="checkbox" \sum_{n=0}^{N-1} 1 = N$$

☐ 所以：

$$\square Z_k[s_k] \approx N A_k e^{j\theta_k} + W_k[s_k]$$

☐ 正确 bin 上的信号是 N 点相干叠加，幅度大，相位也主要由 θ_k 决定。

☐ 但如果 $b \neq s_k$ 并且 $s_k - b$ 是非零整数，那么：

$$\square \sum_{n=0}^{N-1} e^{j2\pi(s_k-b)n/N} = 0$$

☐ 所以：

$$\square Z_k[b] \approx W_k[b]$$

☐ 错误 bin 上没有相干信号主分量，理论上只剩噪声和泄漏。

☐ 这就是你实验图里 selected bin 和 wrong bin 差异巨大的根本原因。

☐ 2. 为什么 selected bin 的相位能 smooth，wrong bin 不行

☐ 正确 bin 上：

$$\square Z_k[s_k] \approx N A_k e^{j\theta_k}$$

☐ 所以：

$$\square \angle Z_k[s_k] \approx \theta_k$$

☐ 如果 θ_k 来自 CFO/global phase accumulation，那么它通常是连续的、低阶的：

$$\square \theta_k \approx \theta_0 + \alpha k$$

☐ 或者带一点 drift：

$$\square \theta_k \approx \theta_0 + \alpha k + \beta k^2$$

☐ 所以 selected bin 的 unwrap phase 会很平滑，甚至 R^2 很高。

☐ 但是 wrong bin 上：

$$\square Z_k[b] \approx W_k[b]$$

☐ 这时候相位是 $\angle W_k[b]$ ，噪声的相位本身没有稳定物理含义。于是 wrong bin 的 wrapped phase 看起来乱，unwrap phase 也不具备稳定轨迹。

☐ 你图里 selected 的曲线是黑色，unwrap phase 基本是一条稳定下降线，而且标注 $R^2 = 0.999$ 、 $ER = 7.79 \times 10^{-1}$ 、 $rank = 1$ 。

☐ 这说明 selected bin 是高能量、强相干、rank 第一的 bin。相位平滑是可信的。

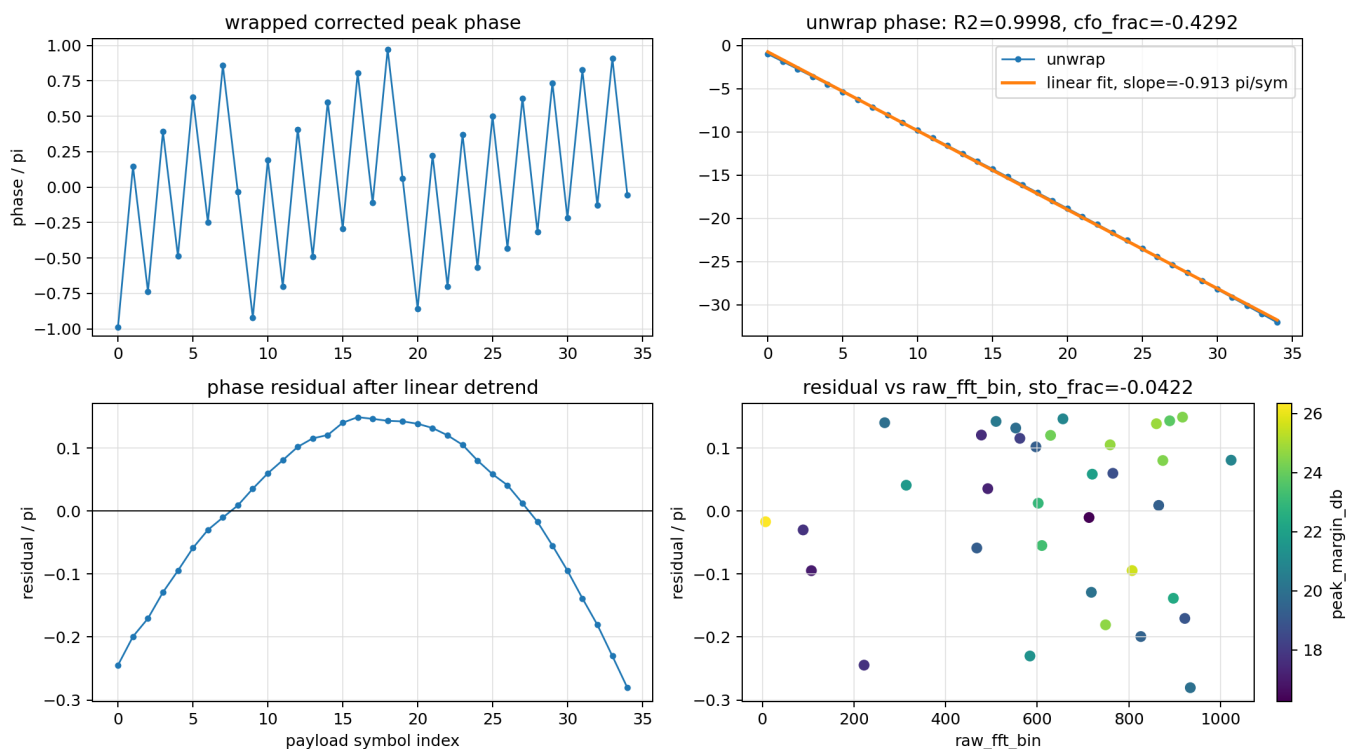
☐ 而 wrong bin 例如 $ER = 5.15 \times 10^{-4}$ 、 $rank = 620$ ，或者

$ER = 2.25 \times 10^{-4}$ 、 $rank = 751$ 。这些能量占比太低，说明它们根本不是有效信号主分量。即使某些 wrong bin 的 unwrap phase 偶尔有较高 R^2 ，也不能说明它有判决价值，因为它的幅度和能量占比低到几乎不可用。

- ☐ 3. 为什么 wrong bin 偶尔也会有一点趋势
- ☐ 你的图里有些 wrong bin 的 unwrap phase 不是完全随机，比如 fixed 000 的 $R^2 = 0.876$ ，但它的能量占比只有 5.15×10^{-4} 。
- ☐ 这说明它可能捕捉到了一点点全局相位泄漏，或者某些频谱泄漏旁瓣里也带着同一个 global CFO phase。但这类相位不可靠，因为它的信号幅度太低。
- ☐ 这个可以用带残余频偏的 Dirichlet kernel 来解释。
- ☐ 如果 corrected 后仍有一点 fractional residual, dechirp 后的 tone 不完全落在整数 bin，而是 $s_k + \delta$ ，那么 FFT bin b 上的信号项变成：
 - ☐ $Z_k[b] \approx A_k e^{j\theta_k} D_N(s_k + \delta - b) + W_k[b]$
- ☐ 其中：
 - ☐
$$D_N(x) = \sum_{n=0}^{N-1} e^{j2\pi x n / N}$$
- ☐ 这是 Dirichlet kernel。
- ☐ 当 $b = s_k$ 时， $|D_N(\delta)|$ 很大，主峰强。当 $b \neq s_k$ 时， $|D_N(s_k + \delta - b)|$ 通常很小，只是旁瓣或泄漏。因此 wrong bin 也可能含有一点与 θ_k 同源的相位，但它很弱，容易被噪声淹没。
- ☐ 所以你的实验结果很合理：wrong bin 不是完全没有任何相位结构，但其 amplitude 和 energy ratio 太低，phase 不具备稳定判决能力。

5.29

四联图分析：



首先右下角说明了，参与相位并不是由STO知道引起的，要不然会呈现相位和raw_fft_bin的线性关系。

其次左下角说明了，这个绝对不可能是SFO残余主导的相位变化，SFO主导的不可能这么平稳。

▼ 实验说明

可以，我现在更确定一点了：左下角那个二次形 residual，大概率不是 SFO 主导，而是 corrected 后残留的 packet-level/global phase drift，最像“残余 CFO 随时间缓慢变化”或硬件相位漂移。

依据不是拍脑袋，是几条证据叠起来的。

1. 如果是 residual STO，应该和 bin 有关，但现在几乎无关

你前面说的公式里，STO 或 SFO 的一部分会和 `s_q` / FFT bin 耦合。那左下角 residual 应该在右下角 `residual vs raw_fft_bin` 里表现出明显相关性。

但你这批数据里，`residual_bin_fit_r2` 大多很低，比如 packet 1 是 `0.00002`，packet 5 是 `0.047`，packet 9 是 `0.046`。也就是说：

```
1 phase residual 不是 raw_fft_bin 的函数
```

这不支持“主要是 residual STO * bin”这个解释。

2. 我做了回归：左下角 residual 几乎被 q^2 单独解释

用 payload symbol index `q` 做了回归，也对 raw_fft_bin 做了回归。多数正常包结果是：

```
1 packet 1:  $q^2$  R2 = 0.994, bin R2 = 0.001
2 packet 2:  $q^2$  R2 = 0.999, bin R2 = 0.006
3 packet 5:  $q^2$  R2 = 0.998, bin R2 = 0.000
4 packet 9:  $q^2$  R2 = 0.999, bin R2 = 0.000
5 packet 13:  $q^2$  R2 = 0.999, bin R2 = 0.002
```

这说明左下角那条曲线更像：

```
1 只随 symbol 时间 q 变化的公共相位项
```

而不是：

```
1 随 symbol bin / payload 内容变化的项
```

所以你怀疑 `q*s_q(a-1)` 那部分不该形成这么稳定的二次曲线，这个判断是对的。

3. 关掉 SFO cursor adjustment，形状几乎不变

我还做了一个小实验：不用 gr-lora 那种 SFO sample insertion/deletion cursor，只按固定 symbol 长度往后切 payload，再重算 corrected FFT。

结果 selected bin 一个都没变， `$q^2$  R2` 基本也不变：

- 1 packet 5: 原 corrected q^2 $R^2=0.998$, 不做 SFO cursor 仍是 0.998
- 2 packet 9: 原 corrected q^2 $R^2=0.999$, 不做 SFO cursor 仍是 0.999
- 3 packet 13: 原 corrected q^2 $R^2=0.999$, 不做 SFO cursor 约 1.000

这说明左下角这条 residual 不是由 gr-lora 的 SFO 逐符号 sample adjust 直接画出来的。

4. 能量没有崩, 说明不是明显的 chirp 内部 SFO 失配

SFO 的含 **r** 项主要会让 chirp 内部相位形状错掉, 表现为 FFT 泄漏、peak margin 下降、energy ratio 下降。研究和 gr-lora_sdr 论文也这么处理: SFO 会造成 STO 随时间累积, 严重时会让 symbol 被误判到相邻 bin, 需要通过丢/复制 sample 补偿。gr-lora_sdr 论文里也明确说 SFO 的主要后果是 STO 随时间线性累积, 可能超过半个 chip 而导致相邻 symbol 错判; 补偿方式是周期性丢弃或复制 sample。来源: Tapparel 等的 gr-lora_sdr 论文, SFO 段落与 demod 说明。

https://events.gnuradio.org/event/24/contributions/641/attachments/192/478/paper_tapparel.pdf

但你这批 corrected selected peak:

- 1 peak_energy_ratio $\approx 0.78 - 0.88$
- 2 peak_margin $\approx 19 - 21$ dB
- 3 rank = 1

能量非常集中。所以它不像“chirp 内部相位形状被 SFO 搞坏了”。

5. 它更像残余 CFO / global phase drift

LoRa 论文里有个很关键的事实: fractional CFO 会造成相邻符号之间的相位线性旋转, gr-lora_sdr 论文也写了 fractional CFO estimator 正是利用 consecutive symbols 的 phase rotation。也就是说, CFO 本来就会直接进入你画的 selected peak phase。

同一篇 gr-lora_sdr 论文还说明 CFO/STO 会影响 dechirp+FFT, 整数 CFO 移动 spectrum, fractional CFO 会把能量扩散到邻近 bin。

你现在看到的是:

- 1 unwrap phase 主体非常线性
- 2 减掉线性后, 剩下 q^2 慢变

这在物理上最像:

- 1 残余频偏不是常数, 而是随时间缓慢漂移

因为相位是频率误差的积分:

- 1 constant residual CFO → phase 线性
- 2 linearly drifting CFO → phase 二次

所以左下角二次 residual 更像：

- 1 residual/global CFO drift
- 2 或硬件 oscillator phase drift / phase noise 的低阶趋势

而不是 SFO 的 bin-dependent 项。

关于信道相位

普通静态信道相位：

$$1 \quad h = |h| e^{j\{\dots\}}$$

只会给整包加一个常数相位，会被线性拟合的 intercept 吃掉，不会留下二次曲线。

但如果是 time-varying channel，比如移动、多径慢变、硬件模拟链相位漂移，它也可能贡献这种低阶 common phase drift。只是从你这批数据看，所有 selected bin 能量都很稳，我会优先怀疑硬件/残余 CFO 漂移，而不是 payload symbol 或 SFO。

最终判断

这张左下角图不能叫“SFO 结论”。更准确应该写成：

- 1 corrected selected peak phase 中存在 packet-level low-order common phase drift;
- 2 该 drift 与 FFT bin 几乎无关，且不显著破坏 FFT 能量集中；
- 3 因此它更像 residual CFO / oscillator phase drift，而不是 SFO/ST0 的 bin-dependent phase error。

后续相位特征要这样用：

不要把二次 residual 当 SFO 证据；可以把它当 common phase drift feature，再结合 energy ratio / margin / rank 判断 bin 是否可信。